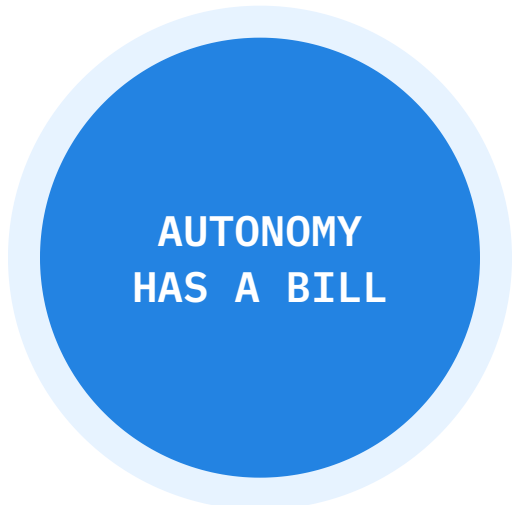


ARCHITECTURE FOUNDATION • 2026 EDITION

What Are Microservices?

แยกระบบอย่างมีเหตุผล ไม่ใช่แยกเพราะ Diagram ดูแพง



AUTONOMY
HAS A BILL

คำตอบสั้นที่สุด

Microservice คือส่วนของระบบที่ดูแล **Business capability** ชัดเจน เป็นเจ้าของข้อมูล และ **Deploy แยก** จากส่วนอื่นได้



คำว่า Micro บวกใบน้อยกว่าคำว่า Independent

จบคอร์สแล้ว เราควรตอบให้ได้

BOUNDARY

ควรตัดตรงไหน

อะไรควรอยู่ Service เดียวกัน และ
อะไรควรแยก

FAILURE

พังแล้วเกิดอะไร

Network, Message และข้อมูลที่ไม่
ได้ Commit พร้อมกัน

DECISION

ควรใช้หรือยัง

เมื่อไร Microservices คู่ม และ
เมื่อไร Monolith ฉลาดกว่า

CHAPTER 01

Microservices คืออะไร

เริ่มจากสิ่งที่มันแก้ ไม่ใช่สิ่งที่มันรันบน

01

จากคนเดียวทำทุกอย่าง สู่หลายสถานี



แยกงานแล้วแต่ละสถานีคล่องขึ้น แต่กระดาษขอเดอร်กลายเป็นปัญหาระบบกระจาย

สี่คุณสมบัติที่ทำให้ Service เป็นอิสระจริง

- 01 **Business capability** รับผิดชอบ Outcome ที่มีความหมาย
- 02 **Independent deployment** Release ได้โดยไม่ลากทุกทีมมาด้วย
- 03 **Data ownership** เป็นเจ้าของ Logic และข้อมูลในขอบเขต
- 04 **Explicit contract** คุยผ่าน API หรือ Event ที่ตกลงกัน

เล็กแค่ไหนถึงเรียก Micro?

เล็กพอให้ทีมหนึ่งดูแลได้
ใหญ่พอให้รับผิดชอบเรื่องหนึ่งได้ครบ



นับบรรทัดไม่ได้บอกว่า Boundary ดีหรือไม่

สามรูปแบบที่ไม่ต้องทะเลาะกัน

Monolith

Build และ Deploy เป็นหน่วยเดียว
เริ่มง่าย Transaction ตรงไปตรงมา

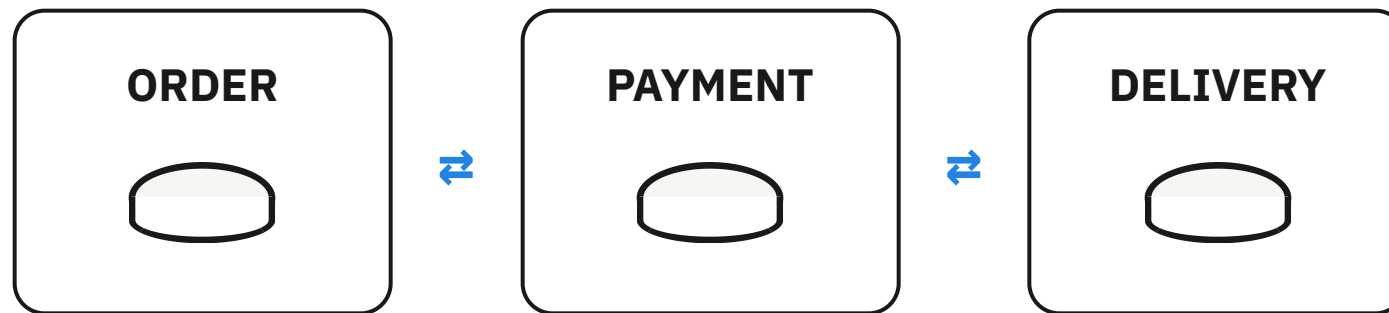
Modular Monolith

หน่วยเดียว แต่แบ่ง Module และ
Ownership ภายในชัด

Microservices

แยก Deploy และข้อมูล แยกกับงาน
ระบบกระจายเต็มรูปแบบ

Distributed Monolith ได้ข้อเสียมาครบสองฝั่ง



Deploy พร้อมกัน · Database เดียว · ล้มตามกัน



Container หลายใบไม่ได้รับประกันความเป็นอิสระ

CHAPTER 02

Service Boundary

ตัดตามธุรกิจ ไม่ใช่ตัดตามชื่อ Folder

02

แยกตามชั้นเทคนิค แล้วทุก Feature ต้องเดินทางไกล

Frontend Service รับ Request

Business Service ตัดสินใจ

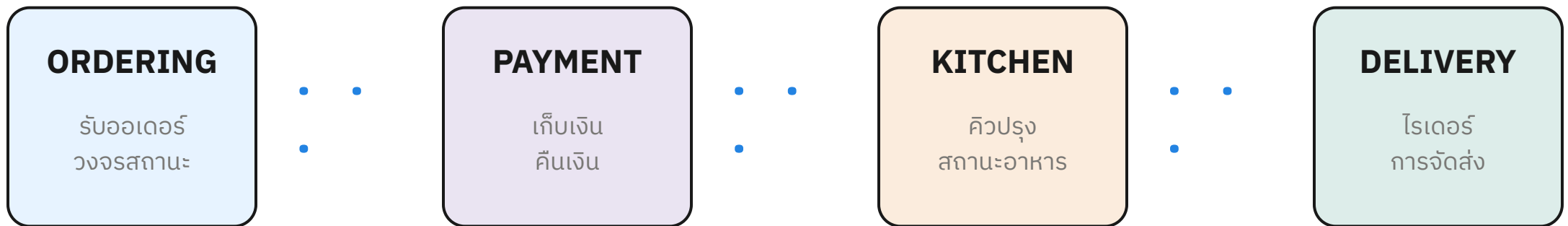
Data Service อ่านเขียนทุก Table

Email Service Deploy ทั้งระบบเพื่อเปลี่ยนกฎเดียว



เราแยกกล่องสำเร็จ แต่ยังแยกการเปลี่ยนแปลงไม่ได้

แยกตามสิ่งที่ธุรกิจทำ

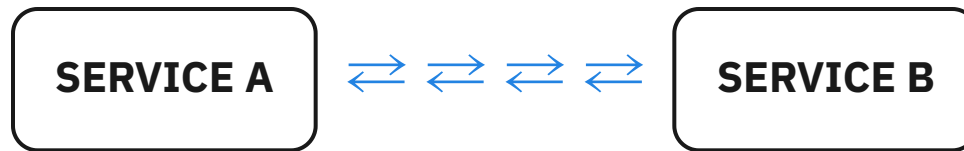


💡 แต่ละ Context มีภาษา กฎ และเจ้าของของตัวเอง

ก่อนแยก ลองถามห้าข้อ

- ✓ **Change together** เปลี่ยนพร้อมกันบอยแคไหน
- ✓ **Chatty calls** ต้องโทรหากันทุกสองบรรทัดหรือไม่
- ✓ **Data owner** ใครเป็น Source of truth
- ✓ **Failure** ล้มแยกกันแล้วระบบทำอะไรต่อ
- ✓ **Team** มีเจ้าของที่ดูแลได้ครบวงจรหรือยัง

ถ้าคุยกันไม่หยุด อาจควรอยู่บ้านเดียวกัน



คุยถี่ · ใช้ข้อมูลเดียว · Release คู่กัน



Merge ก็เป็น Architecture decision ไม่ใช่การถอยหลัง

ทุก Fact ต้องมีบ้านหลักหนึ่งหลัง

SOURCE OF TRUTH

Payment owns payment data

Service อื่นขอผ่าน Contract ไม่เขียน Table แทนเจ้าของ

```
// อย่าทำ
orderingDB.exec(
  "UPDATE payments ..."
)

// ให้ Payment ตัดสินใจ
payment.authorize(order)
```



Schema ภายในไม่ควรกลายเป็น Public API โดยบังเอิญ

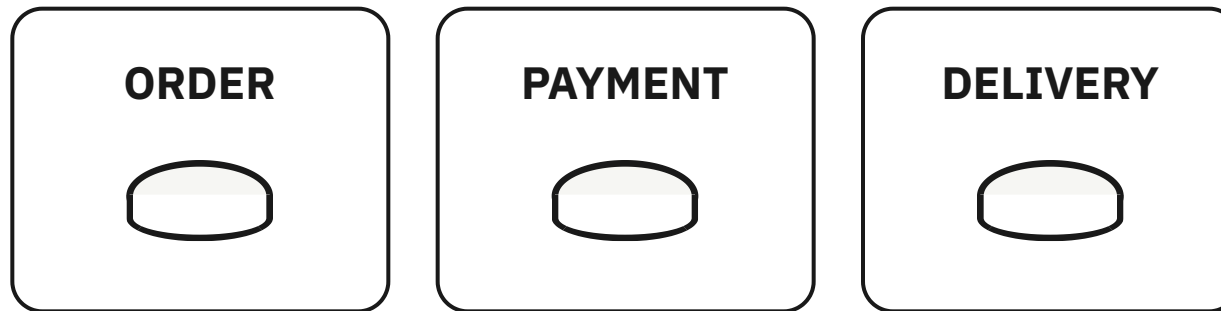
CHAPTER 03

Data & Messages

พอ Database แยก Transaction ก็เริ่มมีเนื้อเรื่อง

03

Database per Service หมายถึง Ownership



แชร์ Physical server ได้ แต่ไม่แชร์ Table และสิทธิ์เขียนกันตามอริยาสัย

ข้อมูลซ้ำได้ ถ้ารู้ว่าใครคือของจริง

CATALOG

ราคา ณ ตอนนี้

฿129

Source of truth สำหรับเมนูปัจจุบัน

ORDER

ราคาตอนซื้อ

฿99

Snapshot ที่ต้องไม่เปลี่ยนตามอนาคต



Eventual consistency ต้องมีเวลาเป้าหมาย วิธี Reconcile และพฤติกรรมระหว่างรอ

Sync หรือ Async เลือกตามสิ่งที่ต้องรอ

Synchronous

ต้องใช้คำตอบทันที

Flow เข้าใจง่าย

Availability และ Latency ผูกกัน

HTTP · gRPC

Asynchronous

ส่งแล้วทำงานต่อได้

Buffer ช่วง Traffic พุ่ง

ต้องรับ Message ช้าและผลที่มาก็หลัง

Queue · Event broker

Event ควรบอกว่าเกิดอะไรขึ้นแล้ว

กว้างจนต้องแกะถุงดู

DataChanged

OrderUpdated

ProcessCompleted

Business fact

OrderPlaced

PaymentAuthorized

DeliveryAssigned

Timeout ไม่ได้บอกว่างานปลายทางล้ม

01

Ordering ส่งคำขอเก็บเงิน 450 บาท

02

Payment เก็บเงินสำเร็จ

03

Network ขาดก่อน Response กลับ

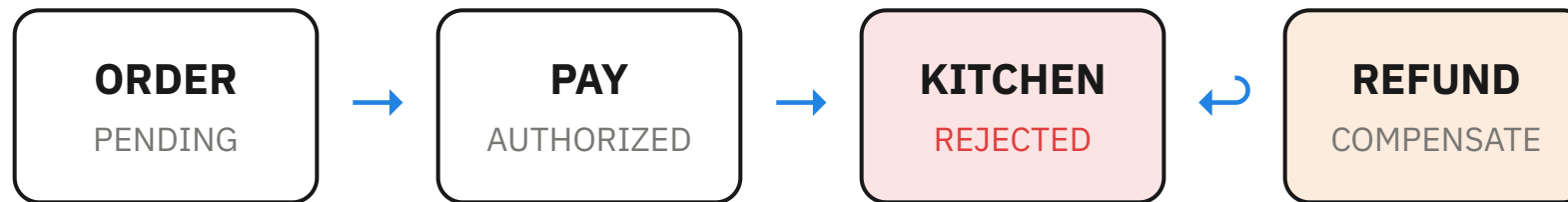
?

Ordering เห็น Timeout แล้วควร Retry หรือไม่



Idempotency key ทำให้ Request เดิมไม่เก็บเงินซ้ำ

Saga เปลี่ยน Transaction ใหญ่เป็นขั้นที่ชดเชยได้



Compensation คือรายการธุรกิจใหม่ ไม่ใช่ย้อนเวลาทั้งระบบ

Outbox ผูกข้อมูลกับ Event ใน Transaction เดียว

```
BEGIN
```

```
  INSERT INTO orders (...)
```

```
  INSERT INTO outbox (...)
```

```
COMMIT
```

Outbox Publisher

อ่านรายการที่ยังไม่ส่ง แล้ว Publish ไป Broker

Idempotent Consumer

รับ Event ซ้ำโดยไม่ทำผลกระทบซ้ำ

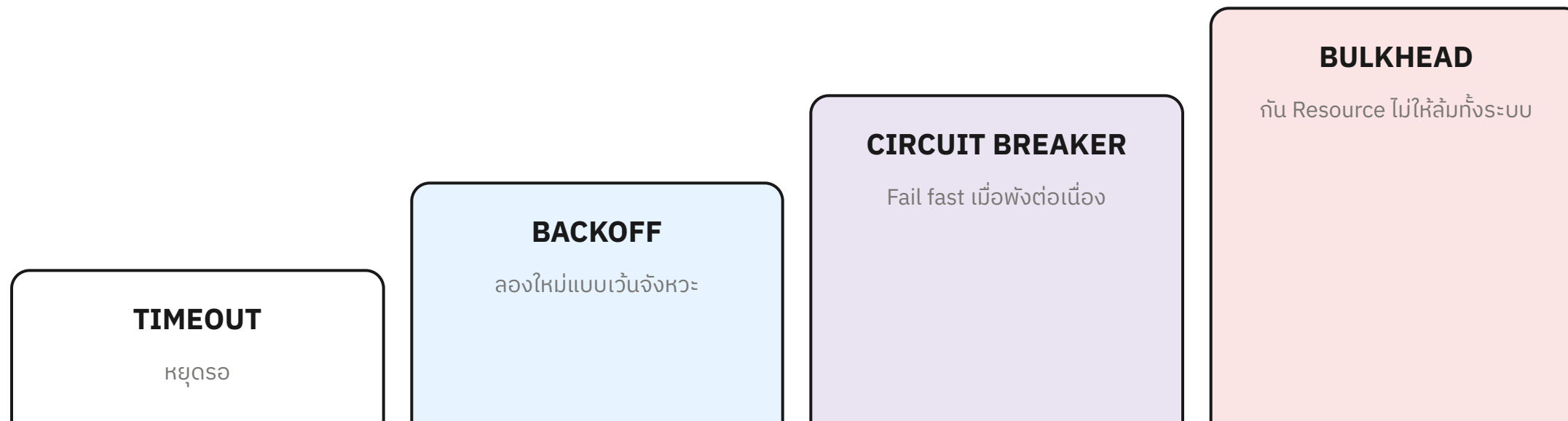
CHAPTER 04

Production Changes the Rules

Network ซ้ำ Service ล่ม และ Message มาเกินหนึ่งครั้ง

04

Resilience ไม่ใช่ Retry จนปลายทางยอมแพ้



 Retry เฉพาะ Transient failure และ Operation ที่ทำซ้ำได้ปลอดภัย

Observability ต้องตาม Request ข้ามบ้านได้



Logs

เกิดเหตุการณ์อะไร



Metrics

เกิดมากแค่ไหน



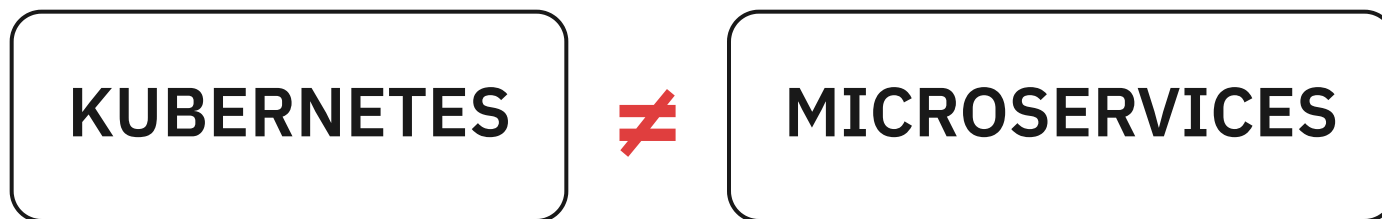
Traces

Request เดินผ่านที่ใด



OpenTelemetry ช่วยส่ง Context และ Signals ด้วยมาตรฐานร่วม

Kubernetes ไม่ได้ทำให้ระบบเป็น Microservices

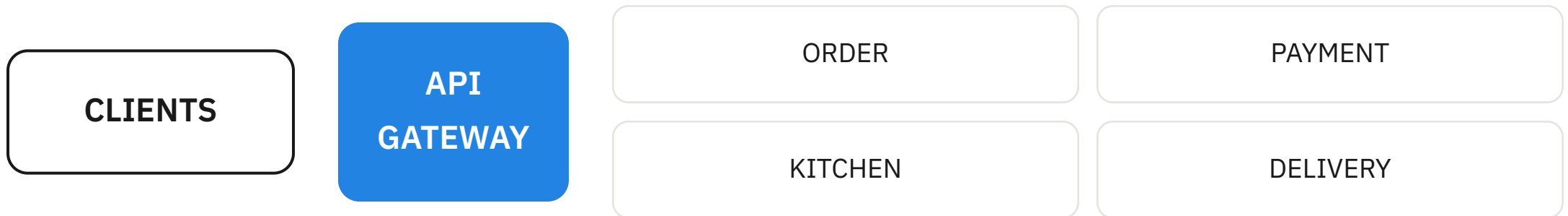


ตัวหนึ่งจัดวาง Workload อีกตัวหนึ่งกำหนด Boundary และ Ownership



กล่องจำนวนมากยังใช้ Database เดียวและ Release พร้อมกันได้

Gateway เป็นประตูหน้า ไม่ใช่กระทรวงทุกอย่าง



Routing, TLS, Auth และ Rate limit อยู่หน้าได้ แต่ Business rule ควรอยู่ใน Domain

Deploy แยกได้ ต้องเปลี่ยน Contract แบบไม่หักคอเพื่อนบ้าน

- ✓ เพิ่ม Field แบบ Optional ก่อน
- ✓ ย้าย Consumer ทีละตัว
- ✓ ตรวจสอบ Breaking change ใน Pipeline
- ✓ รองรับ Code เก่าและใหม่ระหว่าง Rollout
- ✓ ลบของเก่าเมื่อไม่มี Consumer เหลือ

จำนวน Service เพิ่ม จำนวนประตูก็เพิ่ม

Identity

User และ Service ต้องพิสูจน์ตัวตนกับสิทธิ์

Least privilege

Network, Data และ Secret เท่าที่ต้องใช้

Supply chain

ตรวจ Dependency, Image และ Artifact



Gateway รู้ว่า Token จริง แต่ Domain ยังต้องรู้ว่าใครอ่าน Order ของใครได้

CHAPTER 05

Should We Use It?

คำตอบที่ดีอาจเป็น “ยัง”

05

Microservices เริ่มคุ่มเมื่ออิสระมีมูลค่า

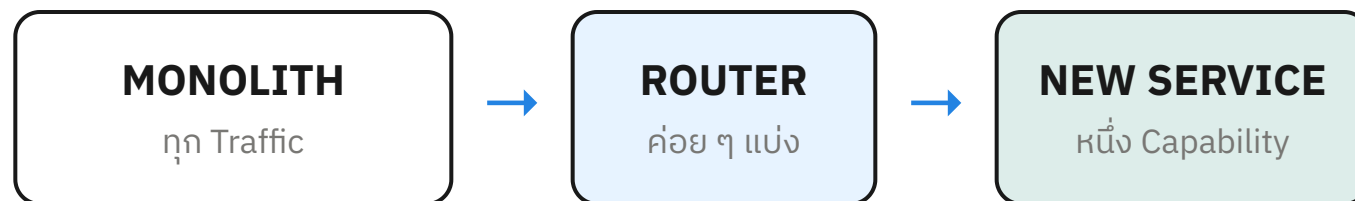
- ✓ หลายทีมต้อง Release คนละจังหวะ
- ✓ บาง Capability ต้อง Scale หรือแยก Fault ต่างกัน
- ✓ Domain และ Ownership เริ่มชัด
- ✓ มี Test, CI/CD และ Observability รองรับ

ถ้ายังเป็นแบบนี้ Modular Monolith มักฉลาดกว่า

- ! ทีมเล็กและ Product ยังเปลี่ยนทิศทุกสัปดาห์
- ! ทุก Feature ต้องแตะทุก Service
- ! ยังไม่มี Automated delivery และ Monitoring
- ! เหตุผลหลักคือบริษัทอื่นมี

🪑 ไม่ต้องเช่ารถบรรทุกสิบล้อเพื่อย้ายเก้าอี้ตัวเดียว

ย้ายทีละ Capability ให้ของเก่ากับใหม่ Coexist



วัดผล · มี Rollback · ปลดของเก่าเมื่อไม่มีใครใช้

Microservices Anti-pattern Bingo

Service ต่อ Function

Shared Database ใครอยากเขียนก็เชิญ

Sync chain หกชั้น

Event ชื่อ DataChanged

Kubernetes ก่อน Monitoring

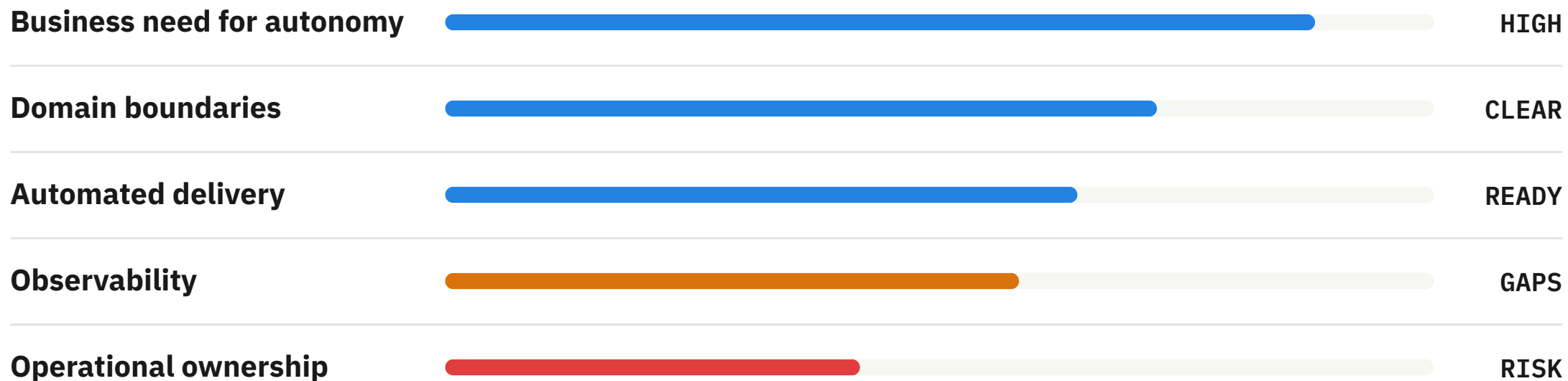
Shared library บังคับ Upgrade พร้อม
กัน

ไม่มี Owner

Retry ไม่มีวันหยุด

หนึ่ง Feature แก่ 17 Repo

Readiness วัดจากระบบและทีม ไม่ใช่จำนวน Pod



อิสระที่ได้ต้องมีค่ามากกว่าค่าประสานงานที่เพิ่ม

ออกแบบร้านส่งอาหารหนึ่ง Flow

Capability:

Source of truth:

Sync / Async:

Failure:

Evidence / SLO:



วาด Happy path หนึ่งเส้น แล้ววาดจุดที่มันขาดอย่างน้อยสองจุด

ONE IDEA TO REMEMBER

Microservices ชื่อความเป็นอิสระ ด้วยค่าประสานงานของระบบกระจาย

ใช้เมื่ออิสระนั้นมีมูลค่ามากกว่าบิล



Architecture ที่ดีช่วยให้ทีมเปลี่ยนสิ่งสำคัญได้ โดยไม่ต้องปลุกทั้งบริษัทมาดู Log